

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 1
по дисциплине: Исследование операций
по теме: Исследование множества опорных планов системы
ограничений задачи линейного программирования (задачи ЛП)
в канонической форме

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Вирченко Ю.В.

Белгород 2024

Цель работы: изучить метод Гаусса-Жордана и операцию замещения, а также освоить их применение к отысканию множества допустимых базисных видов системы линейных уравнений, и решению задачи линейного программирования простым перебором опорных решений.

Задания для подготовки к работе

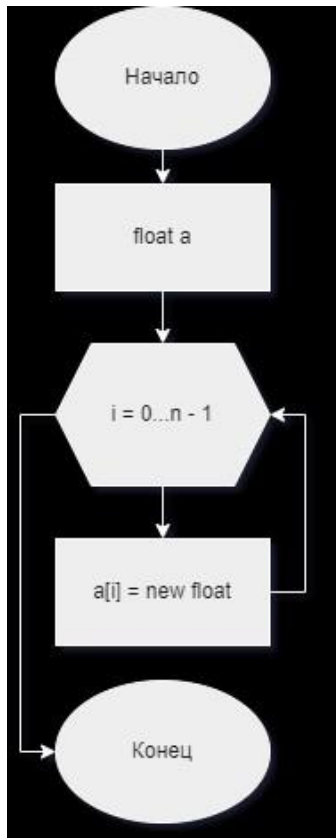
1. Составить программу для отыскания всех базисных видов системы линейных уравнений.
2. Организовать отбор опорных планов среди всех базисных решений, а также нахождение оптимального опорного плана методом прямого перебора. Целевая функция выбирается произвольно.
3. Решить одну из следующих ниже задач вручную (подготовить тестовые данные).

Вариант 7

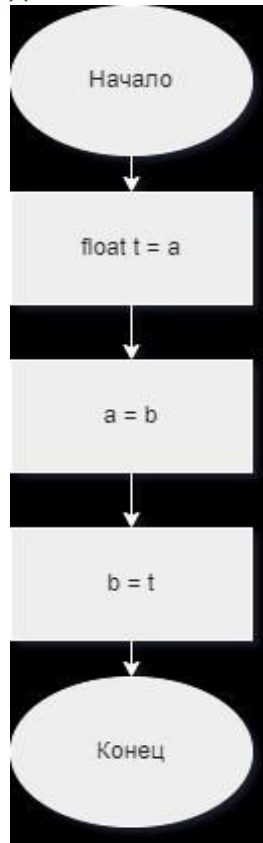
$$7. \begin{cases} -2x_1 + 3x_2 + 4x_3 + 5x_4 + 7x_5 + 10x_6 = 50 \\ 8x_1 + x_2 - 3x_3 + 4x_4 + 5x_5 + 6x_6 = -11 \\ x_1 - 4x_2 + 8x_3 + 9x_4 - 3x_5 - x_6 = 87 \\ 6x_1 + 4x_2 + x_3 + 9x_4 + 12x_5 + 16x_6 = 39 \end{cases}$$

Блок схемы к коду:

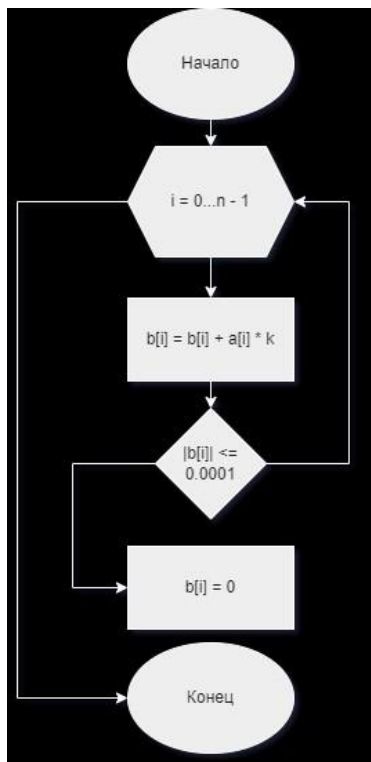
//создание матрицы а размера nxm



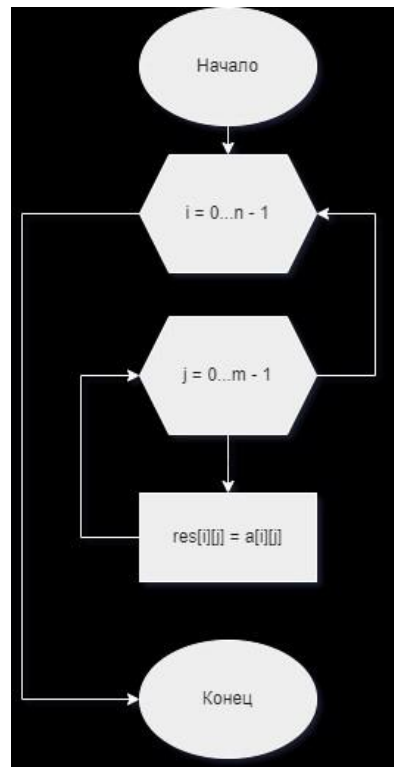
//обмен чисел



//прибавляет к строке b строку a,
умноженную на коэффициент k



//копирование матрицы



```

graph TD
    Start([Начало]) --> InitMax[float max]
    InitMax --> InitMaxI[size_t maxI = 0]
    InitMaxI --> InitCounter[size_t counterOfSwaps = 0]
    InitCounter --> LoopStart{ j = 0...j < n-1 or j < numOfVars }
    LoopStart --> FindMax[max = res[j][j]]
    FindMax --> SetMaxI[maxI = j]
    SetMaxI --> LoopStart
    LoopStart --> LoopInnerStart{i = j + 1...n - 1}
    LoopInnerStart --> CompareMax{res[i][j] > max}
    CompareMax --> SetMax[max = res[i][j]]
    SetMax --> SetMaxIInner[maxI = i]
    SetMaxIInner --> LoopInnerStart
    CompareMax --> LoopInnerStart
    SetMaxIInner --> CompareMaxI{maxI != j}
    CompareMaxI --> Swap[swapArray(res[j], res[maxI], m)]
    Swap --> IncCounter[counterOfSwap = counterOfSwap + 1]
    IncCounter --> LoopInnerStart
    CompareMaxI --> LoopInnerStart
    LoopInnerStart --> LoopInnerCompare{res[i][j]}
    LoopInnerCompare --> CalcK[k = -res[i][j] / res[j][j]]
    CalcK --> ModRow[modRow(res[j], res[i], m, k)]
    ModRow --> LoopInnerStart
    LoopInnerStart --> End([Конец])

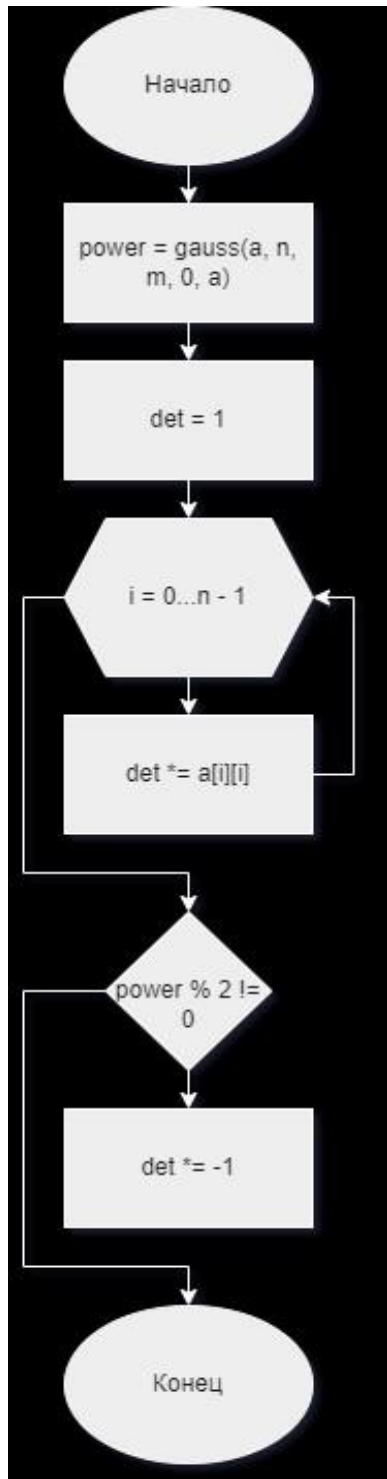
```

```

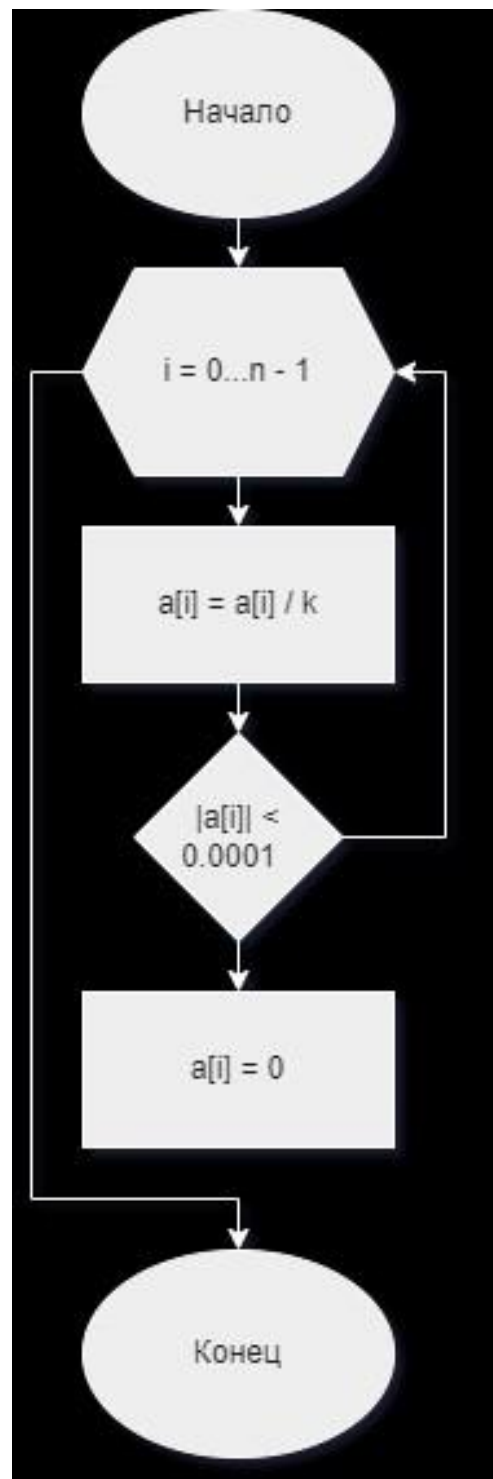
graph TD
    Start([Начало]) --> Init[j = 0, rowsCounter = 0]
    Init --> LoopStart{{i = 0...n - 1}}
    LoopStart --> LoopBody{{j = 0...j < m and (a[i][j] == 0)}}
    LoopBody --> LoopBody
    LoopBody --> LoopStart
    LoopBody --> End([Конец])

```

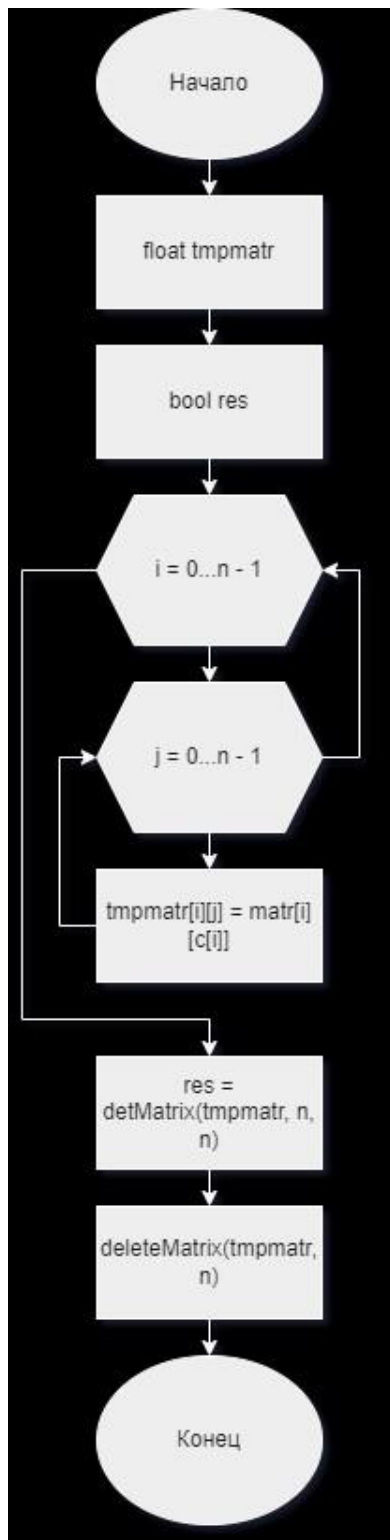
//определитель матрицы



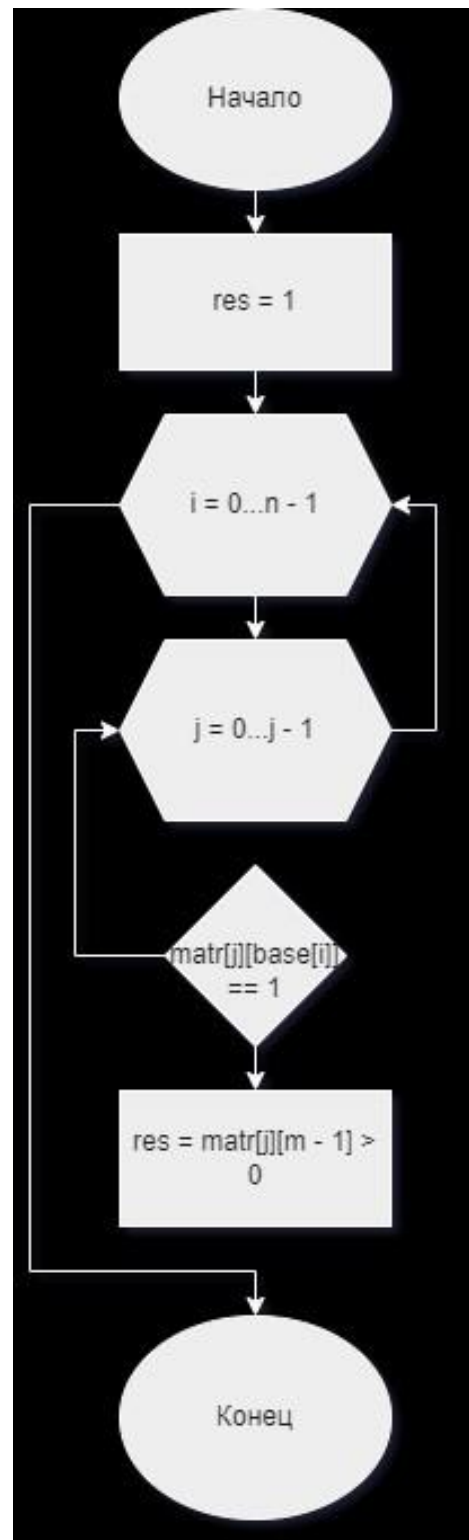
//деление строки на число



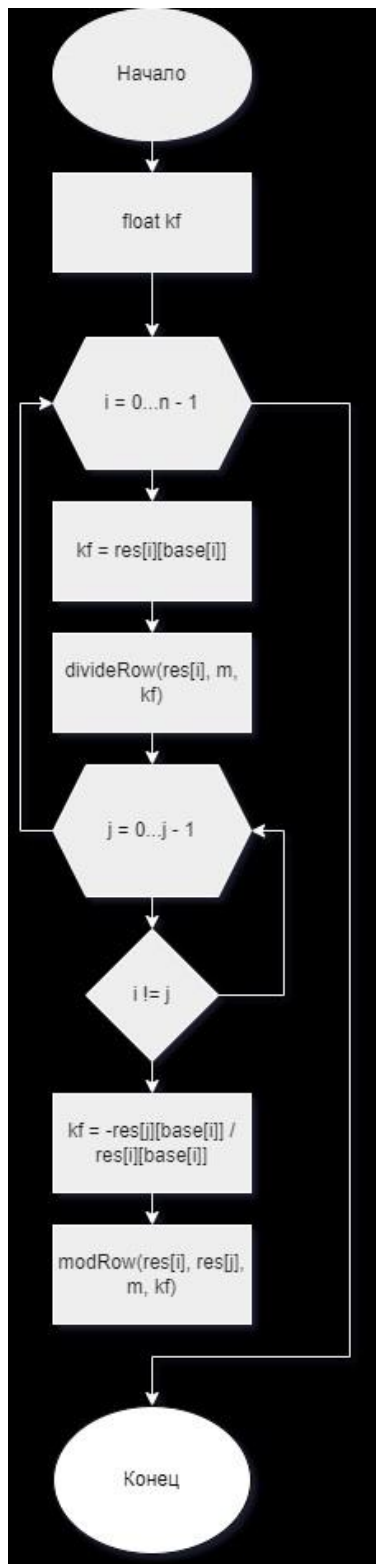
//проверка, можно ли составить
базисное решение из
набора переменных C



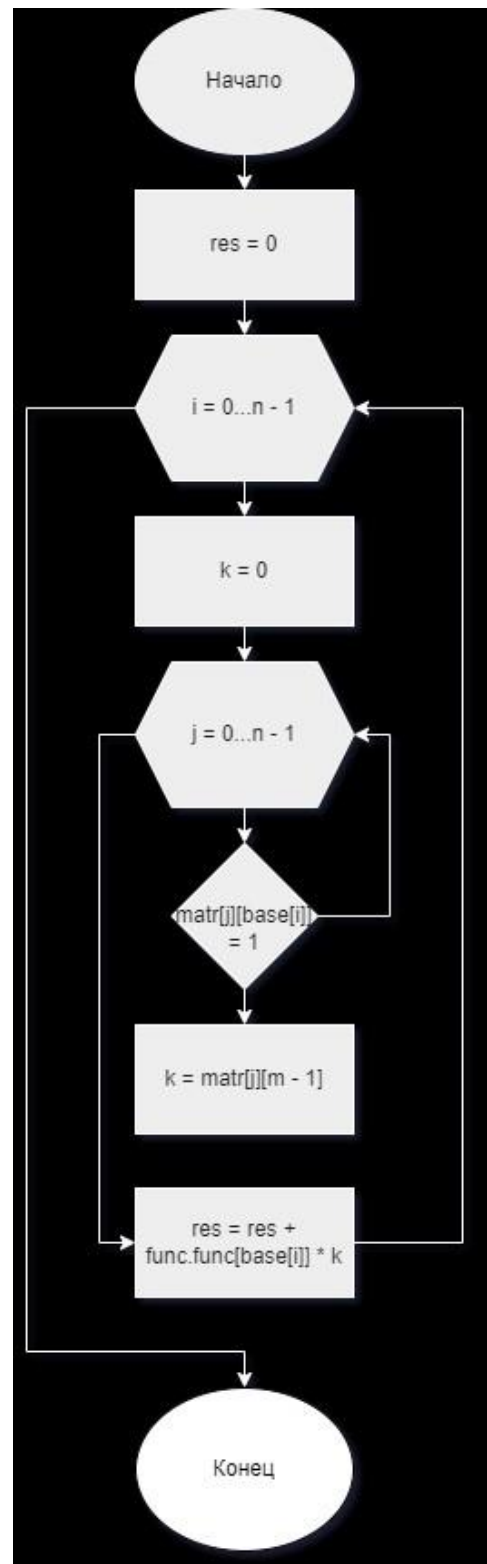
//проверка, является ли базисное решение
опорным планом



//операция замещения базиса



//вычисляет значение целевой функции на данном базисном



```

#include <iostream>
#include <locale>
#include <cmath>
using namespace std;

typedef float** matrix;

typedef struct goalFunc {
    float* func;
    bool toMax;
} goalFunc;

void createMatrix(matrix* a, size_t n, size_t m) {
    *a = new float* [n];
    for (size_t i = 0; i < n; i++)
        (*a)[i] = new float[m];
}

//ввод матрицы а размера nxm
void readMatrix(matrix a, size_t n, size_t m) {
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < m; j++)
            cin >> a[i][j];
}

//вывод матрицы а размера nxm
void writeMatrix(matrix a, size_t n, size_t m) {
    for (size_t i = 0; i < n; i++) {
        for (size_t j = 0; j < m; j++)
            cout << a[i][j] << ' ';
        cout << '\n';
    }
    cout << '\n';
}

//обмен чисел
void Swap(float* a, float* b) {
    float t = *a;
    *a = *b;
    *b = t;
}

//обмен массивов
void swapArray(float* a, float* b, size_t n) {
    for (size_t i = 0; i < n; i++)
        Swap(&a[i], &b[i]);
}

//прибавляет к строке b строку a, умноженную на коэффициент k
void modRow(float* a, float* b, size_t n, float k) {
    for (size_t i = 0; i < n; i++) {
        b[i] += a[i] * k;
        if (abs(b[i]) <= 0.0001)
            b[i] = 0;
    }
}

void copyMatrix(matrix a, size_t n, size_t m, matrix res) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            res[i][j] = a[i][j];
}

//прямой ход Гаусса

```



```

size_t gauss(matrix a, size_t n, size_t m, size_t numOfVars, matrix res) {
    float max;
    size_t maxI, counterOfSwaps = 0;
    // заготавливаем копию матрицы для преобразований
    copyMatrix(a, n, m, res);
    // строим треугольную матрицу
    for (size_t j = 0; (j < n - 1) && (j < numOfVars); j++) {
        // ищем максимум в очередном столбце
        max = res[j][j];
        maxI = j;
        for (size_t i = j + 1; i < n; i++)
            if (abs(res[i][j]) > max) {
                max = abs(res[i][j]);
                maxI = i;
            }
        // меняем максимум
        if (maxI != j) {
            swapArray(res[j], res[maxI], m);
            counterOfSwaps++;
        }
        // выполняем преобразования
        for (size_t i = j + 1; i < n; i++) {
            if (res[i][j]) {
                float k = -res[i][j] / res[j][j];
                modRow(res[j], res[i], m, k);
            }
        }
        //WriteMatr(res, n, m);
    }
    return counterOfSwaps;
}

//подсчёт ненулевых строк матрицы
size_t countRows(matrix a, size_t n, size_t m) {
    size_t j, rowsCounter = 0;
    for (size_t i = 0; i < n; i++) {
        for (j = 0; (j < m) && (a[i][j] == 0); j++);
        rowsCounter += (j < m);
    }
    return rowsCounter;
}

void writeArray(int C[100], int k) {
    for (size_t i = 0; i < k; i++)
        cout << C[i] + 1 << ' ';
    cout << endl;
}

//деление строки на число
void divideRow(float* a, size_t n, float k) {
    for (size_t i = 0; i < n; i++) {
        a[i] /= k;
        if (abs(a[i]) <= 0.0001)
            a[i] = 0;
    }
}

//определитель матрицы
float detMatrix(matrix a, int n, int m) {
    int power = gauss(a, n, m, 0, a);
    float det = 1;
    for (int i = 0; i < n; i++)
        det *= a[i][i];
    if (power % 2 != 0)
        det *= -1;
}

```

```

        return det;
    }

void deleteMatrix(matrix a, size_t n) {
    for (size_t i = 0; i < n; i++)
        delete a[i];
    delete a;
}

//проверка, можно ли составить базисное решение из набора переменных C
bool checkBasic(unsigned* C, int n, matrix matr) {
    float** tmpmatr;
    bool res;
    createMatrix(&tmpmatr, n, n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            tmpmatr[j][i] = matr[j][C[i]];
    res = detMatrix(tmpmatr, n, n);
    deleteMatrix(tmpmatr, n);
    return res;
}

//проверка, является ли базисное решение опорным планом
bool isBasePositive(unsigned* base, int n, int m, matrix matr) {
    bool res = 1;
    for (size_t i = 0; (i < n) && res; i++)
        for (size_t j = 0; (j < n) && res; j++)
            if (matr[j][base[i]] == 1)
                res = (matr[j][m - 1] > 0);
    return res;
}

//операция замещения базиса
void changeBase(matrix a, size_t n, size_t m, unsigned* base, matrix res) {
    copyMatrix(a, n, m, res);
    float kf;
    for (size_t i = 0; i < n; i++) {
        kf = res[i][base[i]];
        divideRow(res[i], m, kf);
        for (size_t j = 0; j < n; j++)
            if (i != j) {
                kf = -res[j][base[i]] / res[i][base[i]];
                modRow(res[i], res[j], m, kf);
            }
    }
}

//вычисляет значение целевой функции на данном базисном решении
float GoalFunc(goalFunc func, unsigned* base, size_t n, size_t m, matrix matr) {
    float res = 0;
    for (size_t i = 0; i < n; i++) {
        float k = 0;
        for (size_t j = 0; (j < n) && !k; j++)
            if (matr[j][base[i]] == 1)
                k = matr[j][m - 1];
        res += func.func[base[i]] * k;
    }
    return res;
}

static void combinationsInner(int i, int b, int n, unsigned* base, float maxV,
goalFunc func, size_t rang, size_t m, float** matr) {
    static unsigned C[100];

```

```

        for (size_t x = b; x <= n - rang + i; x++) {
            C[i] = x;
            if (i == rang - 1) {
                if (checkBasic(C, rang, matr)) {
                    for (size_t lmao = 0; lmao < rang; lmao++)
                        cout << "X" << C[lmao] + 1 << ' ';
                    cout << endl;
                    changeBase(matr, rang, m, C, matr);
                    if (isBasePositive(C, rang, m, matr)) {
                        float funcVal = GoalFunc(func, C, rang, m, matr);
                        if ((funcVal >= maxV) == func.toMax) {
                            maxV = funcVal;
                            for (size_t j = 0; j < rang; j++)
                                base[j] = C[j];
                        }
                    }
                }
            }
            else
                combinationsInner(i + 1, x + 1, n, base, maxV, func, rang, m,
matr);
        }
    }

//составляет все комбинации базовых переменных и выбирает оптимальный план
void combinations(int n, unsigned* base, size_t rang, size_t m, goalFunc func,
float** matr) {
    float maxV = 999;
    if (func.toMax)
        maxV *= -1;
    combinationsInner(0, 0, n, base, maxV, func, rang, m, matr);
}

int main() {
    cout.precision(2);
    size_t n, m;
    setlocale(LC_ALL, "RUS");
    cout << "Введите два числа n и m - размеры матрицы системы ограничений: ";
    cin >> n >> m;
    matrix matrix1, matrix2;

    createMatrix(&matrix1, n, m);
    createMatrix(&matrix2, n, m);

    cout << "Введите матрицу системы ограничений:\n";
    readMatrix(matrix1, n, m);

    cout << "Введённая матрица:\n";
    writeMatrix(matrix1, n, m);

    gauss(matrix1, n, m, m - 1, matrix2);

    cout << "Треугольная матрица:\n";
    writeMatrix(matrix2, n, m);

    size_t rang = countRows(matrix2, n, m);
    size_t varRang = countRows(matrix2, n, m - 1);

    cout << "Ранг расширенной матрицы: " << rang << endl;

    if (rang == m - 1) {
        cout << "У системы есть единственное решение\n";
        return 0;
    } else if (rang != varRang) {
        cout << "Система несовместна\n";
    }
}

```

```

        return 0;
    }

    cout << "Будет " << rang << " базисных переменных и " << m - rang - 1 << "
свободных.\n";

    unsigned* base = new unsigned[rang];
    goalFunc func;
    func.func = new float[rang];

    cout << "Введите целевую функцию - " << m - 1 << " последовательных
чисел:\n";
    for (size_t i = 0; i < m - 1; i++)
        cin >> func.func[i];

    cout << "Введите 0, если значение функции стремится к минимальному, иначе
введите 1: ";
    cin >> func.toMax;

    cout << "Все возможные комбинации базовых переменных: \n";
    combinations(m - 1, base, rang, m, func, matrix2);
    changeBase(matrix2, rang, m, base, matrix2);

    cout << "Оптимальным планом является базисное решение (";
    for (size_t lmao = 0; lmao < rang; lmao++) {
        cout << "X" << base[lmao] + 1 << " = ";
        for (size_t j = 0; (j < n); j++)
            if (matrix2[j][base[lmao]] == 1)
                cout << matrix2[j][m - 1] << ", ";
    }
    cout << "\b\b) " << endl;

    cout << "Значение целевой функции на данном решении: " << GoalFunc(func,
base, rang, m, matrix2);

    deleteMatrix(matrix1, n);
    deleteMatrix(matrix2, n);
}

```

Результат работы программы:

```
Введите два числа n и m - размеры матрицы системы ограничений: 4 7
Введите матрицу системы ограничений:
-2 3 4 5 7 10 50
8 1 -3 4 5 6 -11
1 -4 8 9 -3 -1 87
6 4 1 9 12 16 39
Введённая матрица:
-2 3 4 5 7 10 50
8 1 -3 4 5 6 -11
1 -4 8 9 -3 -1 87
6 4 1 9 12 16 39

Треугольная матрица:
8 1 -3 4 5 6 -11
0 -4.1 8.4 8.5 -3.6 -1.8 88
0 0 9.8 13 5.4 10 1.2e+02
0 0 0 0 0 0 0

Ранг расширенной матрицы: 3
Будет 3 базисных переменных и 3 свободных.
Введите целевую функцию - 6 последовательных чисел:
1 1 1 1 1 1
Введите 0, если значение функции стремится к минимальному, иначе введите 1: 1
Все возможные комбинации базовых переменных:
X1 X2 X3
X1 X2 X4
X1 X2 X5
X1 X2 X6
X1 X3 X4
X1 X3 X5
X1 X3 X6
X1 X4 X5
X1 X4 X6
X1 X5 X6
X2 X3 X4
X2 X3 X5
X2 X3 X6
X2 X4 X5
X2 X4 X6
X2 X5 X6
X3 X4 X6
X3 X5 X6
X4 X5 X6

Оптимальным планом является базисное решение (X3 = 8.1, X4 = 2.6, X6 = 0.5)
Значение целевой функции на данном решении: 11
```

3. Решить одну из следующих ниже задач вручную (подготовить тестовые данные).

Лабораторная работа №1

Вариант 7

$$\begin{cases} -2x_1 + 3x_2 + 4x_3 + 5x_4 + 7x_5 + 10x_6 = 50 \\ 8x_1 + x_2 - 3x_3 + 4x_4 + 5x_5 + 6x_6 = -11 \\ x_1 - 4x_2 + 8x_3 + 9x_4 - 3x_5 - x_6 = 87 \\ 6x_1 + 4x_2 + x_3 + 9x_4 + 12x_5 + 16x_6 = 39 \end{cases}$$

Составлю матрицу для системы

$$\begin{pmatrix} -2 & 3 & 4 & 5 & 7 & 10 & | & 50 \\ 8 & 1 & -3 & 4 & 5 & 6 & | & -11 \\ 1 & -4 & 8 & 9 & -3 & -1 & | & 87 \\ 6 & 4 & 1 & 9 & 12 & 16 & | & 39 \end{pmatrix}$$

Поменяю местами строки для удобства

$$\begin{pmatrix} 8 & 1 & -3 & 4 & 5 & 6 & | & -11 \\ 6 & 4 & 1 & 9 & 12 & 16 & | & 39 \\ \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{matrix} -2 & 3 & 4 & 5 & 7 & 10 & | & 50 \\ 1 & -4 & 8 & 9 & -3 & -1 & | & 87 \end{matrix} \end{pmatrix}$$

Складываем III строку с IV

$$\begin{pmatrix} 8 & 1 & -3 & 4 & 5 & 6 & | & -11 \\ 6 & 4 & 1 & 9 & 12 & 16 & | & 39 \\ \begin{pmatrix} 1 \\ 3 \end{pmatrix} \begin{matrix} -2 & 3 & 4 & 5 & 7 & 10 & | & 50 \\ 0 & -5/2 & 10 & 23/2 & 1/2 & 4 & | & 112 \end{matrix} \end{pmatrix}$$

Смешиваем II строчку с III

$$\begin{pmatrix} -\frac{3}{4} \\ -\frac{1}{4} \end{pmatrix} \left(\begin{array}{cccccc|c} 8 & 1 & -3 & 4 & 5 & 6 & -11 \\ 6 & 4 & 1 & 9 & 12 & 16 & 39 \\ 0 & \frac{13}{3} & \frac{13}{3} & 8 & 11 & 46/3 & 63 \\ 0 & -\frac{5}{2} & 10 & \frac{23}{2} & \frac{1}{2} & 4 & 112 \end{array} \right)$$

Смешиваем I строчку с II

$$\begin{pmatrix} \frac{10}{13} \\ \frac{10}{13} \end{pmatrix} \left(\begin{array}{cccccc|c} 8 & 1 & -3 & 4 & 5 & 6 & -11 \\ 0 & 13/4 & 13/4 & 6 & 33/4 & 23/2 & 189/4 \\ 0 & 13/3 & 13/3 & 8 & 11 & 46/3 & 63 \\ 0 & -5/2 & 10 & 23/2 & 1/2 & 4 & 112 \end{array} \right)$$

Смешиваем II строчку с IV

$$\begin{pmatrix} \frac{1}{3} \\ -\frac{3}{4} \end{pmatrix} \left(\begin{array}{cccccc|c} 8 & 1 & -3 & 4 & 5 & 6 & -11 \\ 0 & 13/3 & 13/3 & 8 & 11 & 46/3 & 63 \\ 0 & 13/4 & 13/4 & 6 & 23/4 & 23/2 & 189/4 \\ 0 & 0 & 25/2 & 419/26 & 89/13 & 167/13 & 3857/26 \end{array} \right)$$

Смешиваем II строчку с III

$$\left(\begin{array}{cccccc|c} 8 & 1 & -3 & 4 & 5 & 6 & -11 \\ 0 & 13/3 & 13/3 & 8 & 11 & 46/3 & 63 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 25/2 & 419/26 & 89/13 & 167/13 & 3857/26 \end{array} \right)$$

Получу единицы на главной диагонали

$$\left(\begin{array}{cccccc|c} 1 & 1/8 & -3/8 & 1/2 & 5/8 & 3/4 & -11/8 \\ 0 & 1 & 1 & 24/13 & 33/13 & 46/13 & 189/13 \\ 0 & 0 & 1 & 419/325 & 178/325 & 334/325 & 3857/325 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right)$$

Цепкая система после преобразований

$$\begin{cases} x_1 = -11/8 - (1/8x_2 - 3/8x_3 + 1/2x_4 + 5/8x_5 + 3/4x_6) \\ x_2 = 189/13 - (x_3 + 24/13x_4 + 33/13x_5 + 46/13x_6) \\ x_3 = 3857/325 - (419/325x_4 + 178/325x_5 + 334/325x_6) \end{cases}$$

Можно заметить, что x_1, x_2, x_3 - базисные переменные, а x_4, x_5, x_6 - свободные. Для получения решения, приравняем

$$x_4, x_5, x_6 = 0$$

$$x_1 = \frac{891}{325} \approx 2,7$$

$$x_2 = \frac{868}{325} \approx 2,9$$

$$x_3 = \frac{3857}{325} \approx 11,8$$

Вывод: Изучив метод Гаусса – Жордана и операции замещения, а так же их применение к нахождению множества базисных видов, Мы решили задачу программно и вручную, получив в результате одинаковые ответы, что даёт понять, что алгоритм решения в обоих случаях верен